



SENTRY

The spending limit your AI agent cannot argue with.

Spending policies for AI agents, enforced by the Canton ledger and readable by nobody but the owner and the agent.

The spending limit on your AI agent is enforced by the same software you're protecting yourself from.

Teams are putting tool-calling agents in front of real money. The caps live in config and a check before submitting, inside the process the agent runs in. Four ordinary failures all end in the same place:

Prompt injection

A bad tool call

A compromised dependency

An ordinary bug

→ a wallet that does what it is told.

On a bad day, the loss is bounded by the balance, not by the policy.

Every other day, the automation nobody switches on, because the only safe setting is a human approving everything.

You choose between a limit you can prove and a limit you can keep private.

	ENFORCED BY SOMETHING THE AGENT CANNOT REACH	LIMITS STAY PRIVATE
Caps in your own orchestration layer	no	yes
Coinbase Spend Permissions · public chain	yes	no
canton-agent-mcp · checked in the wallet app	no	yes
Sentry	yes	yes

A public chain publishes a firm's caps, counterparties and spending cadence. A database or a wallet app enforces the limit but asks everyone to trust whoever runs it. Canton is the one place both properties hold at once.

The owner signs a policy. The ledger enforces it.

WALLETPOLICY • SIGNED BY THE OWNER

Per-transaction cap	200.00
---------------------	--------

Rolling cap • 24 h	300.00
--------------------	--------

Allowlist	merchant
-----------	----------

Auto-approve up to	50.00
--------------------	-------

EXECUTED	40.00 to Merchant	auto-approved
HELD	120.00 to Merchant	above auto-approve, owner signs
HELD	10.00 to Stranger	counterparty not allowlisted
REJECTED	5000.00 to Merchant	insufficient balance

Output of sentry-agent request, an external program holding the agent's own credential.

RequestTransfer is the agent's only choice on the policy. Inside it, payments settle with nobody watching. Outside it, they wait for the owner. Over the balance, they are refused. Every outcome leaves a record.

Canton does the enforcing, the hiding and the sharing.

Daml is the enforcement

Caps, allowlist, rolling window and escalation are checked inside RequestTransfer. No client-side check to patch around.

Sub-transaction privacy

The policy is signed by the owner, observed by the agent, and delivered to no counterparty's node.

Three participants

One synchronizer, one transaction settling across nodes, each node asked what it holds.

Shared control

Releasing or refusing a held request needs 2 of 2 confirmations, through BitSafe's Decentralization Manager.

No single host

The owner is hosted on two participants at threshold 1, and kept spending through a host outage.

CIP-0103 wallets

Discovered through Canton's dApp SDK. The wallet submits as its own party, no actAs.

ISSUE Bank mints a holding	DELEGATE Owner signs the policy	REQUEST Agent asks to pay	DECIDE Settle, hold or refuse	RECORD Every outcome on ledger
-------------------------------	------------------------------------	------------------------------	----------------------------------	-----------------------------------

Delete every check in the agent and nothing changes.

SENTRY-AGENT PROBE

```
REFUSED move the holding directly
REFUSED spend as the owner
REFUSED raise its own caps
REFUSED mint itself a holding
each: DAML_AUTHORIZATION_ERROR
4 of 4 refused by the ledger.
```

Four commands the agent is free to send. Balance 960.00 before and after.

WHO HOLDS THE WALLETPOLICY

NODE	ASKED AS	POLICY
sandbox	owner, agent	held
sidebox	owner (2nd host)	held
pebblebox	bank, merchant	no data
pebblebox	the owner	no data

Asked as the owner's own party, the counterparty node still has no policy. It was never delivered there.

For teams that already let software spend, and can't publish their limits.

SEGMENT	Digital-asset treasury and operations teams, 10 to 500 people, already running unattended payment automation. First on Canton: validator operators, tokenized fund administrators, wallet and payments teams.
USER	The platform engineer who operates the agent, holds its credential and knows their own if statement is the last line of defence.
BUYER	Whoever answers for the money: head of treasury, COO, or the founder.
WHO PAYS	For the operational layer: hosted participants, approvals, policy management across agents, reporting. Per agent per month (hypothesis). The contracts stay Apache-2.0 and free, because anyone must be able to check them.
WHY CANTON	The only ledger where the limit is enforced by something the agent can't reach and still hidden from everyone but the owner and the agent.
NOT FOR	Retail users, consumer assistants spending API credit, and teams happy with public caps: Spend Permissions already serves them.

Distribute through wallets, prove it with a refusal.

CHANNELS

- **The probe as the artefact.** A refusal an engineer ran is pasted into their team's chat. A dashboard is demoed once.
- **CIP-0103 wallets.** Nocturnal and Grofty give users a party and a key but no bounded delegation. We are the missing half, they have the users.
- **Canton ecosystem, directly.** Validators and fund administrators are a small, published list: ten conversations.
- **Open contracts.** Adoption of the Daml is worth more than signups.

PILOT • THREE STEPS • 90 DAYS

1

A real wallet signs. An owner signs a policy from a CIP-0103 wallet we did not write.

Integration: @canton-network/dapp-sdk, already wired

2

A real asset moves. The holding becomes a token-standard asset instead of demo units.

Integration: the Canton token standard

3

A real agent runs for a month. One design partner, one agent, one real cap, on a validator that hosts the owner.

Integration: the partner's participant and their agent's credential. Success is not volume: the held queue caught what it should, and every outcome can be explained from the record.

The mechanism is measured. Adoption isn't yet.

31

Daml scripts pass: 25 tests, 6 setup

7/7

deliberate contract bugs caught by the tests

14/14

claims re-checked on a clean GitHub runner, every push

4/4

agent bypass attempts refused by the ledger

ALSO MEASURED, THREE LIVE PARTICIPANTS

- Policy absent from the counterparty node
- Shared control refused at 1 of 2, settled at 2 of 2
- Owner kept spending with one host offline

NOT YET VALIDATED

0 user interviews. Next: ten ICP conversations, recording the second question asked and whether privacy decides it.

North Star: agent requests the ledger decided for an owner who wasn't watching, per week, counted from the owner's own node.

Claim narrowly. Then ask for the three things that decide it.

NOT BUILT YET

- The asset is self-contained, not a token standard
- The issuer mints on request, like a faucet
- LocalNet only, participants run in memory
- No real wallet extension has signed yet
- One person runs all three nodes, so the operators aren't independent

THE ASK · NOT FUNDING

- 1 **Ten introductions** to people accountable for an agent that already spends.
- 2 **A MainNet validator path**, so a real wallet signs end to end.
- 3 **A token standard to sit on**, so a pilot moves real value.

```
github.com/khalydmaina/sentry · run ./scripts/ci.sh and it prints PASS or FAIL per claim
```

Built during the delivery phase on top of a disclosed pre-hackathon snapshot, ddbdac9. See the README's Provenance section.